



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К КУРСОВОЙ РАБОТЕ

НА ТЕМУ:

«Конструктор из геометрических примитивов»

Студент ИУ7-53Б
(Группа)

(Подпись, дата)

М.А. Семенчук
(И. О. Фамилия)

Руководитель курсовой работы

(Подпись, дата)

Д.А. Шибанова
(И. О. Фамилия)

2026 г.

СОДЕРЖАНИЕ

ОПРЕДЕЛЕНИЯ	4
ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ	5
ВВЕДЕНИЕ	6
1 Аналитический раздел	7
1.1 Трассировка лучей	7
1.1.1 Алгоритм прямой трассировки лучей	7
1.1.2 Алгоритм обратной трассировки лучей	8
1.1.3 Выбор алгоритма	9
1.2 Анализ моделей освещения	9
1.2.1 Модель освещения Ламберта	10
1.2.2 Модель освещения Фонга	12
1.2.3 Модель освещения Блинна-Фонга	13
1.3 Геометрические примитивы и алгоритмы пересечения	15
1.3.1 Сфера	15
1.3.2 Плоскость	16
1.3.3 Треугольник	17
1.3.4 Полигональные модели	17
2 Конструкторский раздел	18
2.1 Алгоритм построения изображений	18
2.2 Алгоритм обратной трассировки лучей	20
3 Технологический раздел	22
3.1 Требования к ПО	22
3.2 Средства реализации	22
3.3 Диаграмма классов	23
3.4 Листинги кода	23
3.5 Графический интерфейс пользователя	28
4 Исследовательский раздел	30
4.1 Анализ производительности ПО	30

ЗАКЛЮЧЕНИЕ	34
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	36
ПРИЛОЖЕНИЕ А К работе прилагается презентация, состоящая из 12 слайдов.	37

ОПРЕДЕЛЕНИЯ

В настоящей расчетно-пояснительной записке применяют следующие термины с соответствующими определениями.

Рендеринг — процесс получения изображения по модели с помощью компьютерной программы

ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

В настоящей расчетно-пояснительной записке применяют следующие сокращения и обозначения.

ПО — программное обеспечение

GI — global illumination (глобальное освещение)

PBR — Physically based rendering (физически-корректный рендеринг)

ЦП (CPU) — центральный процессор (central procesesing unit)

ОЗУ (RAM) — оперативное запоминающее устройство (random access memory)

ГП (GPU) — графический процессор (graphics procesesing unit)

ОС (OS) — операционная система (operating system)

SFML — Simple and Fast Multimedia Library

GLFW — Graphics Library Framework

SDL — Simple DirectMedia Layer

ВВЕДЕНИЕ

Компьютерная графика стала неотъемлемой частью современного мира и с каждым годом находит свое применения все в новых аспектах жизни. Она применяется в кинематографе, игровой индустрии, архитектуре, медицине и других областях.

В последние годы в компьютерной графике активно используется алгоритм трассировки лучей. В связи с этим производители видеокарт начали добавлять в свои устройства ядра, специализирующиеся на трассировке лучей (RT-ядра). Разработчики графических API и библиотек также активно добавляют в свои продукты поддержку этой технологии. Известными примерами могут стать DirectX Ray Tracing (DXR), Vulkan RTX, CUDA/OptiX и другие. Многосторонняя поддержка этой технологии позволяет создавать все более фотореалистичные изображения.

Целью данной работы является разработка ПО для визуализации трехмерных сцен, состоящих из геометрических примитивов, а также проведение исследования по оценке производительности разработанного ПО.

Для достижения поставленной цели необходимо выполнить следующие **задачи**:

- выбрать алгоритм генерации изображения, модель освещения и затенения;
- описать доступные для размещения на сцене модели и формализовать их;
- построить схемы алгоритмов;
- выбрать средства реализации (платформу, язык программирования, графическую библиотеку, GUI библиотеку);
- реализовать спроектированное ПО с помощью выбранных средств;
- провести исследование производительности разработанного ПО.

1 Аналитический раздел

1.1 Трассировка лучей

Трассировка лучей (от англ. Ray tracing)[1] — это метод рендеринга, который позволяет реалистично имитировать освещение среды за счет использования физически корректной модели поведения света. Распространяясь по сцене и взаимодействуя с ее объектами, свет может отражаться, преломляться, поглощаться и рассеиваться.

1.1.1 Алгоритм прямой трассировки лучей

Главная идея, лежащая в основе алгоритма прямой трассировки лучей, заключается в том, что наблюдатель видит объекты посредством испускаемого источниками света, который многократно отражаясь, согласно законам оптики, доходит до глаз наблюдателя. Метод прямой трассировки предполагает построения траекторий лучей от всех источников освещения ко всем точкам всех объектов сцены, лучи отражаются, преломляются или проходят сквозь него и в результате достигает наблюдателя. Если объект не является отражающим или прозрачным, то траектория луча на этой точке обрывается.

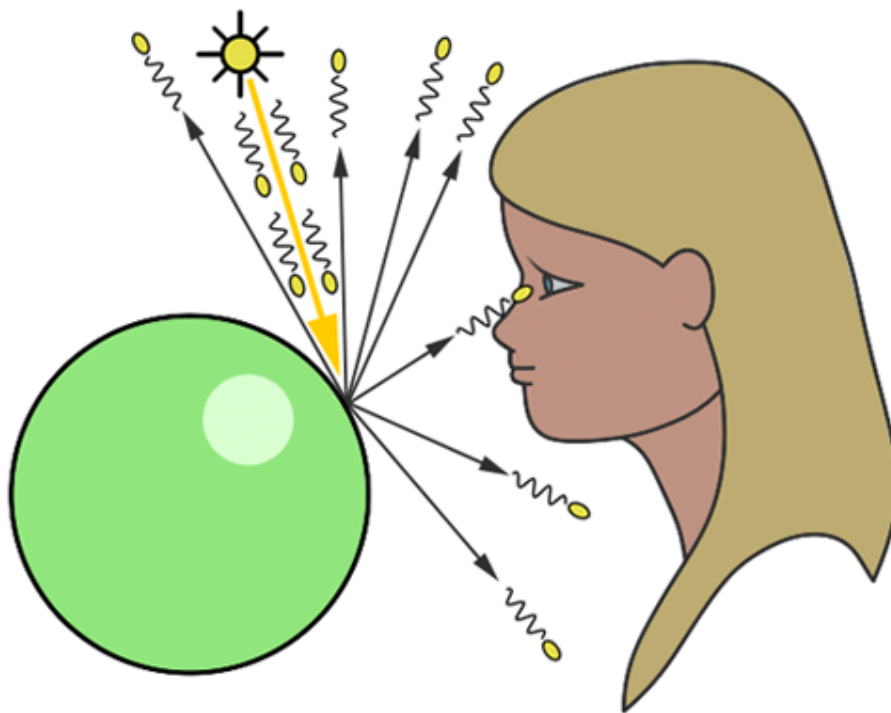


Рисунок 1.1 – Алгоритм прямой трассировки лучей[2]

Основным недостатком этого алгоритма является излишне большое число рассматриваемых лучей, приводящее к существенным затратам вычислительных мощностей, так как лишь малая часть лучей достигает точки наблюдения. Однако данный алгоритм имитирует реальное поведение света и применяется в задачах, в которых точность важнее скорости (оптика и фотоника, расчет освещенности помещения и тд).

1.1.2 Алгоритм обратной трассировки лучей

На отображение сцены влияют только лучи, достигшие наблюдателя. В силу закона обратимости светового пути[3], согласно которому луч света, распространившийся по определенной траектории в одном направлении, повторит свой ход при распространении и в обратном направлении, можно искусственно поменять направления лучей на противоположное, и испускать лучи из точки наблюдения. Это позволяет рассматривать исключительно лучи, которые гарантированно попадут в камеру. Данный метод, основанный на испускании лучей от камеры к объектам сцены, называется обратной трассировкой лучей. Данный подход позволяет существенно сократить количество обрабатываемых лучей и обеспечить более эффек-

тивное формирование изображений при приемлемой степени физической достоверности.

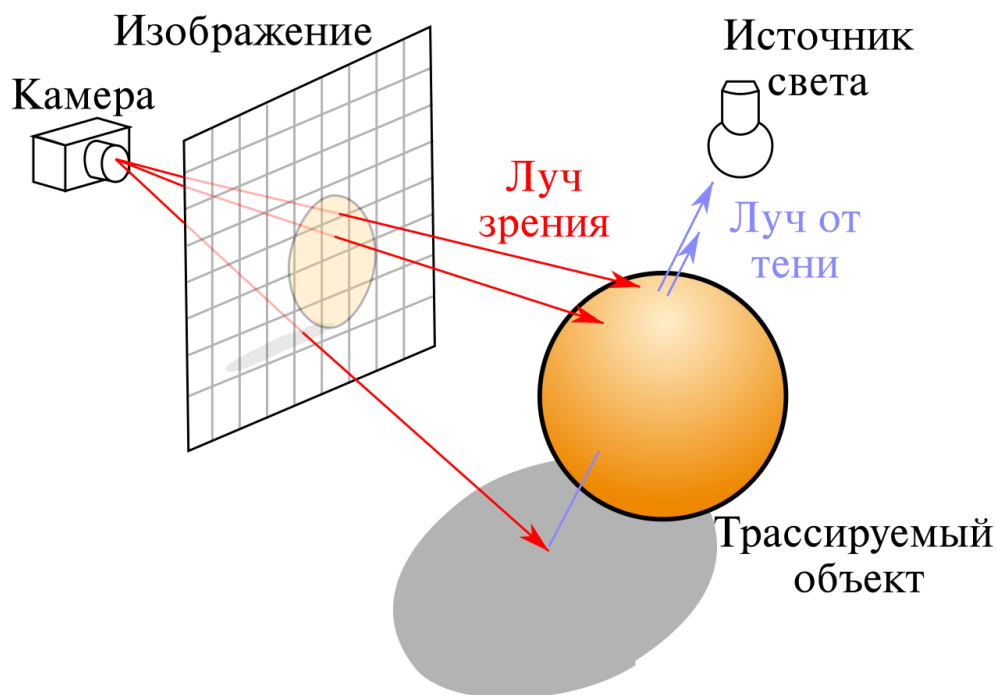


Рисунок 1.2 – Алгоритм обратной трассировки лучей[4]

1.1.3 Выбор алгоритма

В качестве алгоритма рендеринга был выбран метод обратной трассировки лучей, так как для поставленной задачи не требуется полная физическая корректность модели, а также важна производительность, учитывая, что программа будет выполняться на центральном, а не графическом процессоре.

1.2 Анализ моделей освещения

Свет при взаимодействии с поверхностью может быть отражен, поглощен или преломлен. Часть световой энергии поглощается поверхностью и преобразуется в тепло. Отраженный и преломленный свет делают объект видимым для наблюдателя. Если весь падающий свет будет поглощен поверхностью объекта, он будет невидимым (абсолютное черное тело).[5]

Все модели освещения делятся на два вида: глобальные и локальные. Глобальные модели учитывают возможности отражения и преломления света от объектов, не являющихся прямыми источниками освещения, поэтому они требуют значительных вычислительных затрат. Напри-

мер, прозрачные поверхности должны позволять просматривать объекты за ними, на поверхностях возможно появление таких явлений, как рефлекс. Глобальные модели освещения основываются на том, что видимость и затенения связаны между собой: яркость точки поверхности определяется распределением яркости по всем остальным поверхностям, видимым из этой точки. Так можно моделировать шероховатость и свойства отражения реальных материалов, освещение многократно отраженным светом и связанные с ним цветовые эффекты.

Существуют более простые, локальные модели освещения, которые учитывают только свет от источника. Они вычисляют интенсивность, цвет и дальнейшее распределение света на поверхности, но не учитывают перенос света между объектами. Изображение формируется в результате отражения, падающего на поверхность объектов света, интенсивность и цвет которого необходимо рассчитать. Фоновое освещение определяет цвет объекта в тени или при отсутствии явных источников освещения. Его интенсивность постоянна, а также равномерно распределена по всему пространству. Выделяют две основные модели локального освещения: модель Ламберта и модель Фонга.

1.2.1 Модель освещения Ламберта

Модель Ламберта описывает идеальную «матовую» или диффузно отражающую поверхность. Интенсивность отраженного света подчиняется закону косинусов Ламберта, согласно которому отраженное излучение одинаково во всех направлениях. При этом положение и угол обзора наблюдателя не имеют значения, т.к. диффузно отраженный свет рассеивается равномерно во всех направлениях.[5]

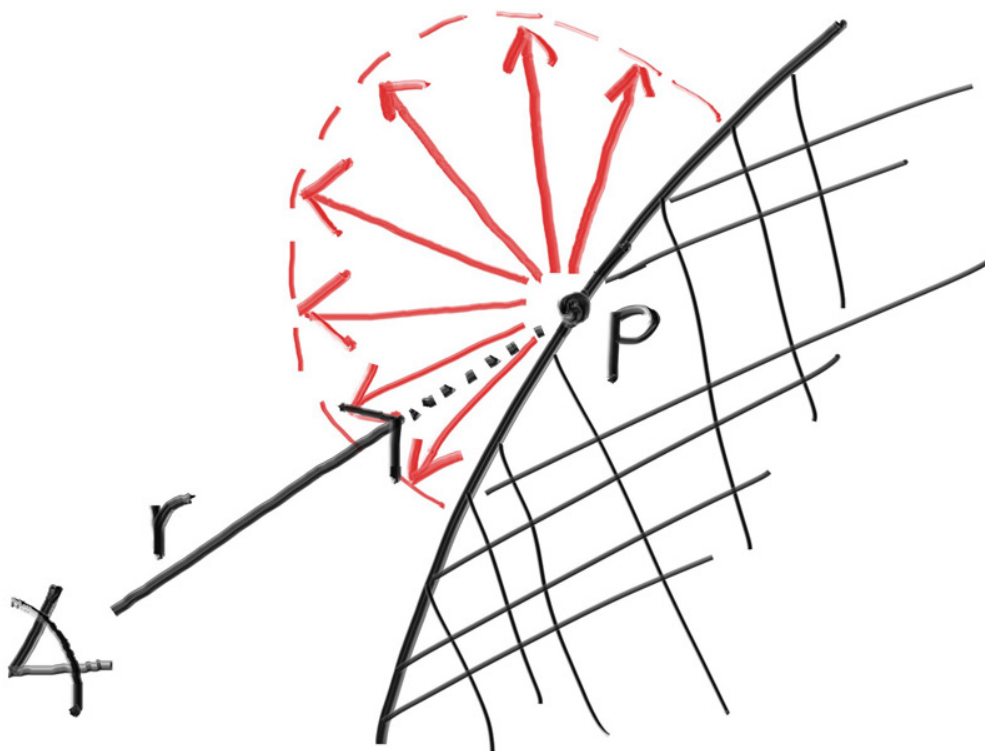


Рисунок 1.3 – Схема диффузного отражения Ламберта[6]

Пусть:

- $\vec{N}$ — вектор нормали к поверхности;
- $\vec{L}$ — вектор от точки до источника;
- K_d — коэффициент диффузного освещения;
- I_0 — интенсивность источника;
- I — результирующая интенсивность света в точке.

Тогда формула расчета освещенности будеи иметь следующий вид:

$$I = I_0 \cdot K_d \cdot (\vec{L} \cdot \vec{N}) \quad (1.1)$$

Из формулы (1.1) следует главный недостаток модели Ламберта - одинаковая интенсивность всех точек, принадлежащих одной грани.

Модель Ламбрета является одной из самых простых моделей освещения. Данная модель часто используется в составе других, более сложных, моделей освещения, т.к. практически в любой другой модели можно выделить диффузную составляющую. Модель Ламберта представляется более-

менее равномерной частью освещения и описывает ориентацию поверхностей сцены.

1.2.2 Модель освещения Фонга

Модель освещения Фонга[7] представляет собой комбинацию диффузной составляющей (модель Ламберта) с зеркальной составляющей. Кроме равномерного освещения диффузного отражения, на объектах появляются блики за счет зеркального отражения. Падающий и отраженный лучи лежат в одной плоскости с нормалью к отражающей поверхности в точке падения, и эта нормаль делит угол на 2 равные части как показано на рисунке 1.4.

Зеркальная составляющая освещенности в точке зависит от того, насколько близки направления на наблюдателя и отраженного луча. Также в модели освещения Фонга используется фоновая составляющая, которая прибавляется к интенсивности в каждой точке. Она является грубым приближением лучей света, рассеянных соседними объектами, а затем достигшими заданной точки.

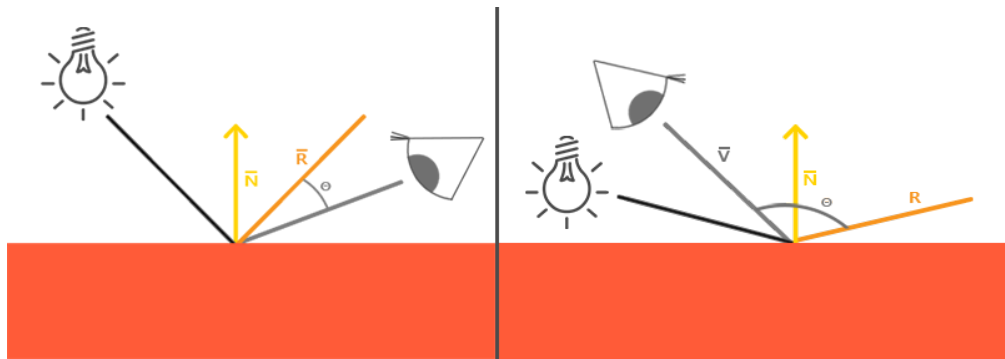


Рисунок 1.4 – Модель освещения Фонга[8], где

- $\vec{N}$ — вектор нормали к поверхности;
- $\vec{V}$ — вектор от точки до наблюдателя;
- $\vec{R}$ — вектор отраженного света.

Пусть:

- K_s — коэффициент зеркального отражения;
- I_0 — интенсивность источника;

- α — коэффициент резкости блика;
- I_s — результирующая зеркальная составляющая света в точке.

Тогда зеркальная составляющая в модели Фонга рассчитывается следующим образом:

$$I_s = I_0 \cdot K_s \cdot (\vec{R} \cdot \vec{V})^\alpha \quad (1.2)$$

Согласно модели Фонга интенсивность в точке будет рассчитываться по формуле:

$$I = I_a + I_d + I_s, \quad \text{где} \quad (1.3)$$

I_a — фоновая составляющая,

I_d — диффузная составляющая,

I_s — зеркальная составляющая

Подставив уравнение каждой из трех составляющих, получим следующую формулу:

$$I = I_0 \cdot K_a + I_0 \cdot K_d \cdot (\vec{L} \cdot \vec{N}) + I_0 \cdot K_s \cdot (\vec{R} \cdot \vec{V})^\alpha \quad (1.4)$$

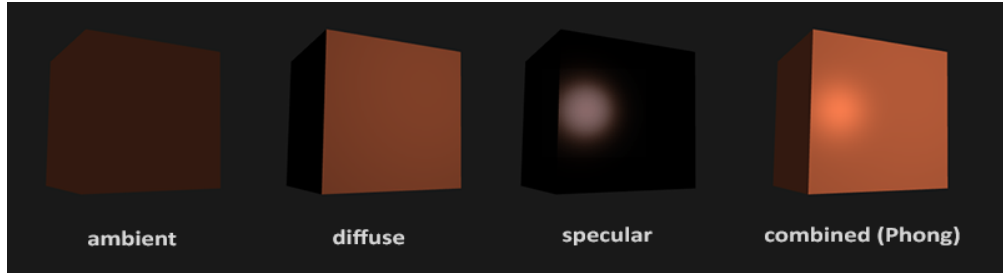


Рисунок 1.5 – Компоненты модели освещения Фонга[9]

1.2.3 Модель освещения Блинна-Фонга

Модель освещения Блинна-Фонга[10] является модификацией модели Фонга, в которой зеркальная составляющая расчиывается следующим способом:

$$I_s = I_0 \cdot K_s \cdot (\vec{N} \cdot \vec{H})^\beta \quad (1.5)$$

Вектор $\vec{H}$ (рис. 1.6) является нормализованной суммой векторов направления на источник $\vec{L}$ и направления на наблюдателя $\vec{V}$, его также

называют полувектором.

$$\vec{H} = \frac{\vec{L} + \vec{V}}{\|\vec{L} + \vec{V}\|} \quad (1.6)$$

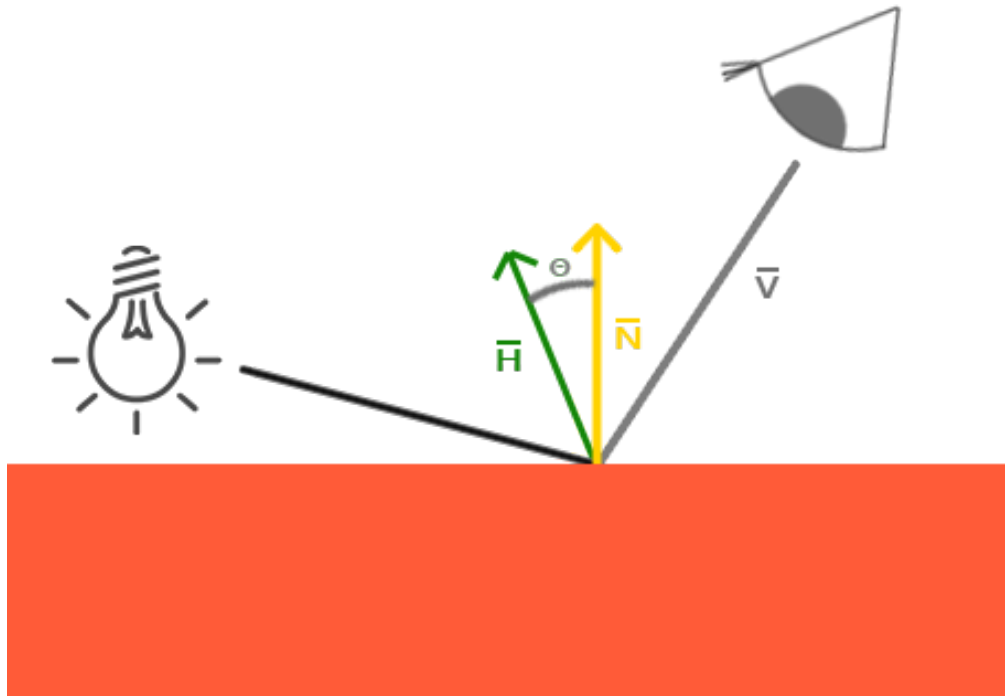


Рисунок 1.6 – Модель освещения Блинна-Фонга[11], где

- $\vec{N}$ — вектор нормали к поверхности;
- $\vec{V}$ — вектор от точки до наблюдателя;
- $\vec{H}$ — срединный вектор между вектором наблюдения и источником.

Основным преимуществом модификации Блинна-Фонга является лучшая производительность за счет упрощенного вычисления зеркальной составляющей, т.к. полувектор $\vec{H}$ вычисляется проще вектора отраженного света $\vec{R}$. Фоновая и диффузная составляющая данной модели рассчитываются аналогично модели Фонга.

В данной работе будет использована модель освещения Фонга, так как она максимально реалистично описывает диффузное и зеркальное отражение света.

1.3 Геометрические примитивы и алгоритмы пересечения

В данном разделе будут описаны способы задания рассматриваемых геометрических примитивов, а также способы пересечения луча с ними.[12] [13]

1.3.1 Сфера

Сфера задается следующими параметрами:

- центром $\vec{C} = (C_x, C_y, C_z)$;
- радиусом r .

Каноническое уравнение сферы имеет вид:

$$(x - C_x)^2 + (y - C_y)^2 + (z - C_z)^2 = r^2$$

Параметрическое уравнение луча имеет вид:

$$\vec{P}(t) = \vec{O} + \vec{D} \cdot t$$

, где $\vec{O}$ — радиус-вектор начала луча, а $\vec{D}$ — вектор направления луча.

Тогда уравнение сферы можно выразить через скалярное произведение векторов $\vec{P}$ и $\vec{C}$:

$$(\vec{P} - \vec{C}) \cdot (\vec{P} - \vec{C}) = r^2$$

Подставив вместо $\vec{P}$ уравнение луча и раскрыв скобки, получим:

$$\vec{D} \cdot \vec{D} \times t^2 + 2 \times (\vec{O} - \vec{C}) \cdot \vec{D} \times t + (\vec{O} - \vec{C}) \cdot (\vec{O} - \vec{C}) - r^2 = 0$$

Перед нами квадратное уравнение относительно переменной t со сле-

дующими коэффициентами:

$$\begin{aligned}a &= \vec{D} \cdot \vec{D} \\b &= 2 \times (\vec{O} - \vec{C}) \cdot \vec{D} \\c &= (\vec{O} - \vec{C}) \cdot (\vec{O} - \vec{C}) - r^2\end{aligned}$$

Учитывая, что вектор направления луча $\vec{D}$ нормализован, коэффициент $a = 1$.

Вычисляем дискриминант по формуле $D = b^2 - 4c$.

- $D < 0 \implies$ пересечений нет;
- $D = 0 \implies$ одно пересечение;
- $D > 0 \implies$ два пересечения.

$$t_{1,2} = \frac{-b + \sqrt{D}}{a}$$

Отрицательные корни ($t < 0$) означают пересечение не самого луча с объектом, а его продолжения (прямой, на которой лежит луч). Ближайшее пересечение соответствует наименьшему положительному корню уравнения.

1.3.2 Плоскость

Каноническое уравнение плоскости имеет вид:

$$Ax + By + Cz + D = 0$$

Причем коэффициенты A , B и C одновременно не равны нулю.

В векторной форме уравнение плоскости записывается следующим образом:

$$(\vec{r} \cdot \vec{N}) + D = 0$$

Подставляя вместо радиус-вектора $\vec{r}$ уравнение луча получаем:

$$((\vec{O} + \vec{D}t) \cdot \vec{N}) + D = 0$$

Используя свойство линейности скалярного произведения:

$$\vec{O} \cdot \vec{N} + t(\vec{D} \cdot \vec{N}) + D = 0$$

Выражаем t :

$$t = -\frac{\vec{O} \cdot \vec{N} + D}{(\vec{D} \cdot \vec{N})}$$

- Если знаменатель равен нулю, то луч параллелен плоскости;
- Если числитель равен нулю, то луч лежит в плоскости;
- Если $t < 0$, то с плоскостью пересекается продолжение луча;
- Если $t > 0$, то луч пересекается с плоскостью.

1.3.3 Треугольник

Задача нахождения пересечения луча с треугольником может быть разделена на 2 подзадачи:

1. Определение точки пересечения луча с плоскостью треугольника;
2. Проверка принадлежности точки пересечения треугольнику.

Для проверки принадлежности точки треугольнику могут быть использованы барицентрические координаты; однако чаще применяется менее ресурсоемкий метод Моллера-Трумбера[14].

1.3.4 Полигональные модели

Остальные геометрические примитивы, такие как параллелепипед, правильная призма и правильная пирамида, представляют собой полигональные модели. Пересечение луча с ними определяется как пересечение с ближайшей гранью, при этом каждая грань рассматривается в виде треугольника.

2 Конструкторский раздел

В этом разделе будут рассмотрены алгоритмы, выбранные для построения изображения, а также спроектированная диаграмма классов приложения.

2.1 Алгоритм построения изображений

В качестве средства описания алгоритма построения изображения будет использоваться диаграмма IDEF0.

Построение изображений в компьютерной графике в контексте диаграммы IDEF0 определяется следующими параметрами:

Описание входных данных

- Объекты определяются геометрией, положением и ориентацией и свойствами материала (цвет, текстуры, нормали);
- Камера определяется проекционными параметрами (перспективная, ортографическая), положением и ориентацией;
- Источники освещения определяются типом (направленный, точечный и др.) , положением и ориентацией (для направленных), цветом и интенсивностью.

Описание элементов управления

- Алгоритм рендеринга: растеризация, трассировка лучей, трассировка пути, ray marching;
- Параметры рендеринга: разрешение получаемого изображения, максимальное число переотражений луча, количество сэмплов на пиксель;
- Модель освещения: локальная (Фонг, Блинн-Фонг) или глобальная (PBR[15]);
- Способ затенения: простое, по Гуро, по Фонгу.

Описание механизмов

- Аппаратное обеспечение (CPU, RAM, GPU + VRAM);

- Графический API (OpenGL, Vulkan, DirectX, Metal и др.);
- Библиотека для окна и ввода (GLFW, SDL, SFML и др.).
- Язык программирования (C++, Python, C#, Java и др.);

Выходными данными является сгенерированный кадр.

Общий алгоритм генерация изображений представлен на рисунке 2.1.



Рисунок 2.1 – IDEF0 диаграмма общего алгоритма построения изображения

Выбранный для реализации алгоритм построения изображений с конкретными методами, моделями и инструментами представлен на рисунке 2.2.

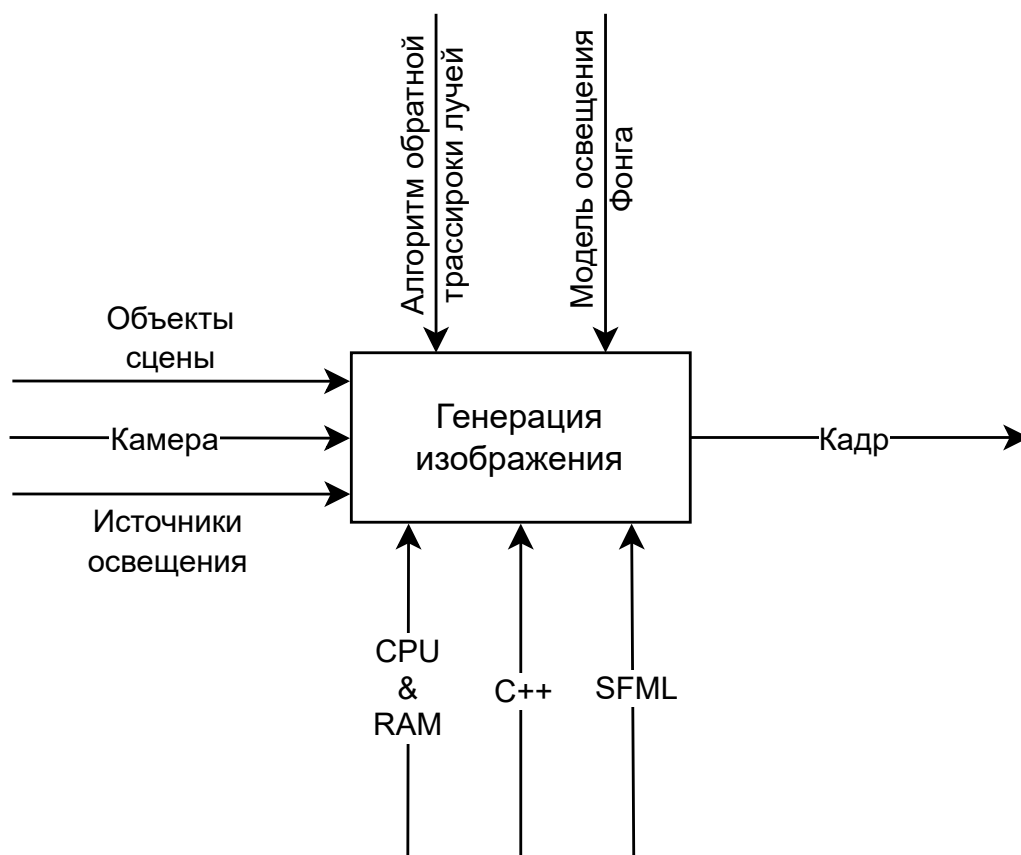


Рисунок 2.2 – IDEF0 диаграмма верхнего уровня (А-0) выбранного для реализации алгоритма построения изображения

2.2 Алгоритм обратной трассировки лучей

IDEF0 диаграмма уровня А-1 алгоритма трассировки лучей представлена на рисунке 2.3.

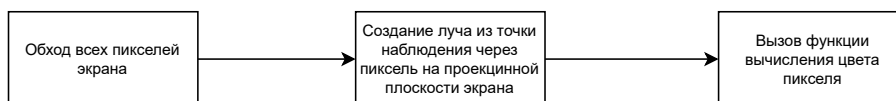


Рисунок 2.3 – IDEF0 диаграмма трассировки лучей

Схема алгоритма обратной трассировки лучей приведена на рисунке 2.4.

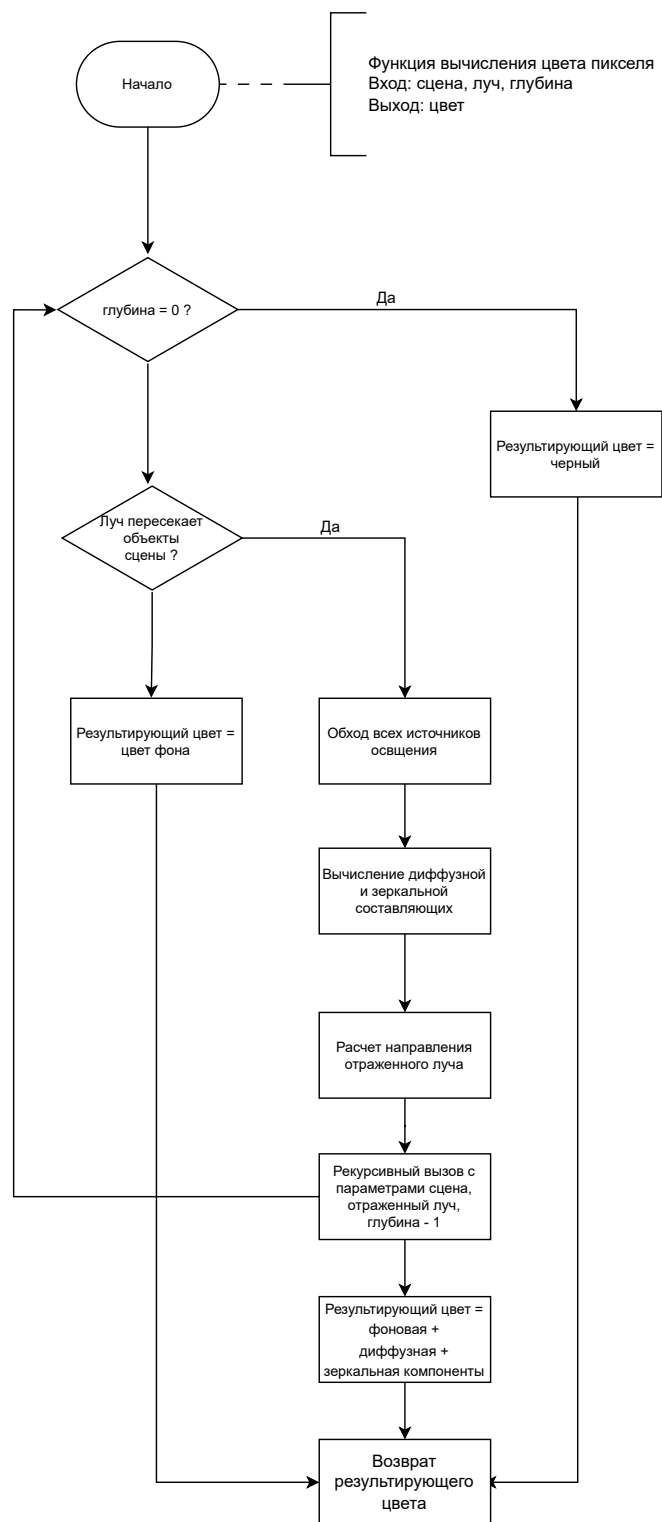


Рисунок 2.4 – Схема алгоритма обратной трассировки лучей

3 Технологический раздел

3.1 Требования к ПО

Разрабатываемое ПО должно обладать следующим функционалом:

- добавление и удаление объектов;
- изменение положения, ориентации, масштаба и цвета объектов;
- изменение положения и ориентации камеры;
- добавление и удаление источников освещения;
- изменение положения, ориентации и цвета источников освещения;

При запуске программы должна создаваться камера.

3.2 Средства реализации

Для написания данной курсовой работы был выбран C++. Выбор языка программирования обусловлен следующими факторами:

- высокая производительность;
- эффективное управления памятью;
- поддержка ООП;
- развитая экосистема графических библиотек;
- поддержка параллельных вычислений;
- личный опыт использования.

В качестве графической библиотеки была выбрана SFML[16] по причине поддержки объектно-ориентированной модели, что отличает ее от GLFW[17], SDL[18].

Для реализации графического интерфейса пользователя был использован Dear ImGui[19] по причине легковесности, быстродействия и интеграции с вышеперечисленными библиотеками для создания окон и ввода.

Для генерации файлов сборки проекта был использован CMake[20].

3.3 Диаграмма классов

Диаграмма классов в нотации UML приведена на рисунке 3.1.

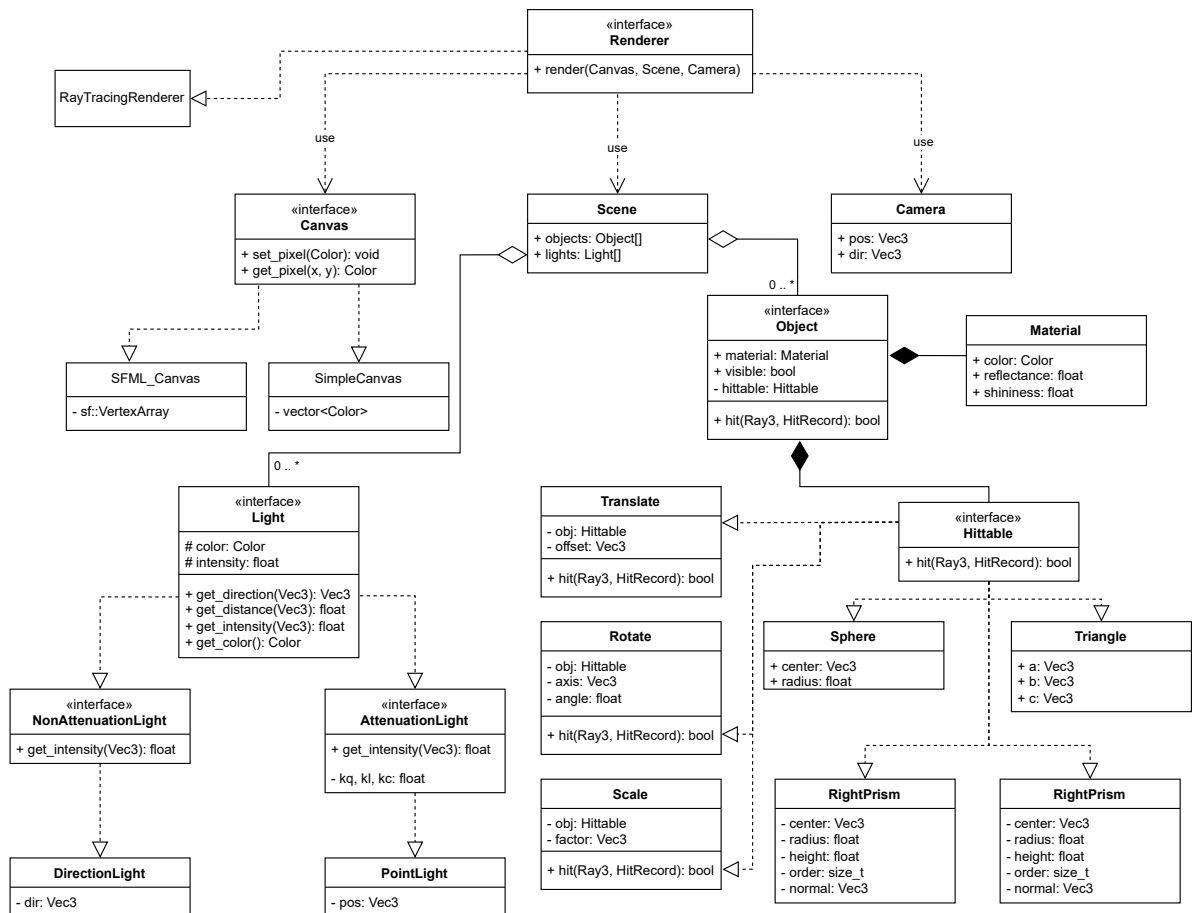


Рисунок 3.1 – Диаграмма классов приложения

Класс *Scene* предназначен для хранения геометрических объектов сцены и источников освещения. Класс *Camera* инкапсулирует логику работы камеры, включая ее положение и ориентацию. Класс *Canvas* используется для сохранения сгенерированного изображения. Класс *Renderer* осуществляет формирование изображения сцены (*Scene*) с позиции камеры (*Camera*) и записывает результат в выходной буфер (*Canvas*). Этот класс выполняет роль центрального модуля, обеспечивая взаимодействия всех остальных компонентов системы.

3.4 Листинги кода

В этом разделе будут приведены листинги основных классов и функций программы.

На листинге 3.1 представлен метод `hit` класса `RayTracingRenderer`. Метод осуществляет обход всех пикселей кадра, вычисляет их цвет и записывает его на холст.

Листинг 3.1 – Метод `render` класса `RayTracingRenderer`

```
1 void RayTracingRenderer::render(Canvas &canvas, const Scene
   &scene, const Camera &camera, const RenderSettings
   &settings) override {
2     float view_height = 2 * std::tan(camera.fov_y / 2) *
       camera.near;
3     float view_width = view_height * camera.aspect;
4
5     size_t width = canvas.get_width();
6     size_t height = canvas.get_height();
7
8     omp_set_num_threads(settings.count_threads);
9
10    #pragma omp parallel for schedule(dynamic)
11    for (int row = 0; row < height; ++row) {
12        float ndc_y = 1 - (row + 0.5f) / height * 2;
13        float dy = ndc_y * view_height / 2;
14        for (int col = 0; col < width; ++col) {
15            float ndc_x = (col + 0.5f) / width * 2 - 1;
16            float dx = ndc_x * view_width / 2;
17            Vec3 ray_dir = (dx * camera.right + dy * camera.up
              + camera.dir).normalized();
18            Ray3 ray(camera.pos, ray_dir);
19            Color color = trace_ray(scene, ray,
              settings.max_ray_bounces);
20            canvas.set_pixel(row, col, color.as_srgb());
21        }
22    }
23 }
```

На листинге 3.2 представлен метод `trace_ray` класса `RayTracingRenderer`. Метод рекурсивно вычисляет цвет пикселя.

Листинг 3.2 – Метод `trace_ray` класса `RayTracingRenderer`

```
1 Color RayTracingRenderer::trace_ray(const Scene &scene, const
   Ray3 &ray, size_t depth) const {
2     if (depth == 0)
3         return Color();
4 }
```



```

5     HitRecord closest_hit;
6     auto closest_obj = find_closest_obj(closest_hit, scene,
7         ray);
8     if (!closest_obj) {
9         return scene.get_background_color(ray);
10    }
11
12    const Vec3 N = closest_hit.normal;
13    const Vec3 V = -ray.direction;
14
15    Color diffuse_intensity;
16    Color specular_intensity;
17
18    for (const auto &light : scene.lights) {
19        if (!is_in_shadow(scene, closest_hit.point, *light)) {
20            const Vec3 R = 2 * N.dot(L) * N - L;
21            specular_intensity += std::pow(std::max(R.dot(V),
22                0.0f), closest_obj->material.shininess) *
23                light->get_color() *
24                light->get_intensity(closest_hit.point);
25        }
26    }
27
28    Color diffuse_total = diffuse_intensity * (1 -
29        closest_obj->material.reflectance) *
30        closest_obj->material.color;
31    Color specular_total = specular_intensity *
32        closest_obj->material.reflectance;
33
34    Vec3 reflected_dir = ray.direction - 2 *
35        N.dot(ray.direction) * N;
36    Ray3 reflected_ray(closest_hit.point + reflected_dir *
37        EPSILON, reflected_dir);
38    Color reflective_color = trace_ray(scene, reflected_ray,
39        depth - 1) * closest_obj->material.reflectance;
40
41    Color final_color = diffuse_total + specular_total +
42        reflective_color;
43    return final_color;
44 }

```

Класс Hittable представляет собой интерфейс, от которого наследуют-

ся все геометрические примитивы сцены. Каждая производная реализация обязана предоставлять метод `hit`, отвечающий за проверку пересечения луча с объектом. Класс `Object` объединяет геометрический каркас объекта (`Hittable`) и его материал (`Material`), который определяет цвет объекта и отражающие способности поверхности. Кроме того, класс содержит метаданные, такие как `visible`, определяющие видимость объекта на сцене. Оба класса представлены на следующем листинге 3.3.

Листинг 3.3 – Классы `Hittable` и `Object`

```
1 class Hittable {
2     public:
3         virtual bool hit(const Ray3 &ray, HitRecord &hit_record)
4             const = 0;
5         virtual ~Hittable() = default;
6 };
7
8 class Object {
9     public:
10        Object(const std::shared_ptr<Hittable> &hittable, const
11            Material &material, bool visible = true)
12            : hittable(hittable), material(material),
13              visible(visible) {}
14
15        bool hit(const Ray3 &ray, HitRecord &hit_record) const {
16            return hittable->hit(ray, hit_record);
17        }
18
19        std::shared_ptr<Hittable> get() {
20            return hittable;
21        }
22
23        Material material;
24        bool visible;
25
26    private:
27        std::shared_ptr<Hittable> hittable;
28 };
```

На листинге 3.4 представлен метод `hit` класса `Sphere`, вычисляющий пересечение луча со сферой.

Листинг 3.4 – Метод `hit` класса `Sphere`

```

1 bool Sphere::hit(const Ray3 &ray, HitRecord &hit_record) const
2 {
3     Vec3 oc = ray.origin - center;
4     auto dot = oc.dot(ray.direction);
5     auto D = dot * dot - oc.dot(oc) + radius * radius;
6     if (D < 0)
7         return false;
8     auto t2 = -dot + std::sqrt(D);
9     if (t2 < 0)
10         return false;
11     auto t1 = -dot - std::sqrt(D);
12     auto t_min = t1 > 0 ? t1 : t2;
13     hit_record.dist = t_min;
14     hit_record.point = ray.at(t_min);
15     hit_record.normal = (hit_record.point -
16         center).normalized();
17     return true;
18 }

```

На листинге 3.5 представлен метод `hit` класса `Triangle`, вычисляющий пересечение луча с треугольником.

Листинг 3.5 – Метод `hit` класса `Triangle`

```

1 bool Triangle::hit(const Ray3 &ray, HitRecord &hit_record)
2     const {
3     constexpr Real EPSILON =
4         std::numeric_limits<Real>::epsilon();
5
6     auto edge_ab = b - a;
7     auto edge_ac = c - a;
8
9     auto pvec = ray.direction.cross(edge_ac);
10    auto det = edge_ab.dot(pvec);
11
12    if (std::abs(det) < EPSILON)
13        return false;
14
15    auto inv_det = 1 / det;
16    auto tvec = ray.origin - a;
17    auto u = inv_det * tvec.dot(pvec);
18    if (u < 0 || u > 1)
19        return false;
20
21    auto t = tvec.dot(edge_ab) * inv_det;
22    if (t < 0 || t > 1)
23        return false;
24    auto v = 1 - u - t;
25    if (v < 0 || v > 1)
26        return false;
27    auto point = a + edge_ab * u + edge_ac * t;
28    auto dist = ray.at(t).dist;
29    hit_record.dist = dist;
30    hit_record.point = point;
31    hit_record.normal = (point - a).normalized();
32    return true;
33 }

```

```

18     auto qvec = tvec.cross(edge_ab);
19     auto v = inv_det * ray.direction.dot(qvec);
20     if (v < 0 || u + v > 1)
21         return false;
22
23     auto t = inv_det * edge_ac.dot(qvec);
24     if (t < EPSILON) {
25         return false;
26     }
27     hit_record.dist = t;
28     hit_record.point = ray.at(t);
29     auto normal = edge_ab.cross(edge_ac).normalized();
30     hit_record.normal = normal.dot(ray.direction) < 0 ? normal
31         : -normal;
32     return true;
}

```

3.5 Графический интерфейс пользователя

На рисунке 3.2 приведен пример работы графического интерфейса пользователя.

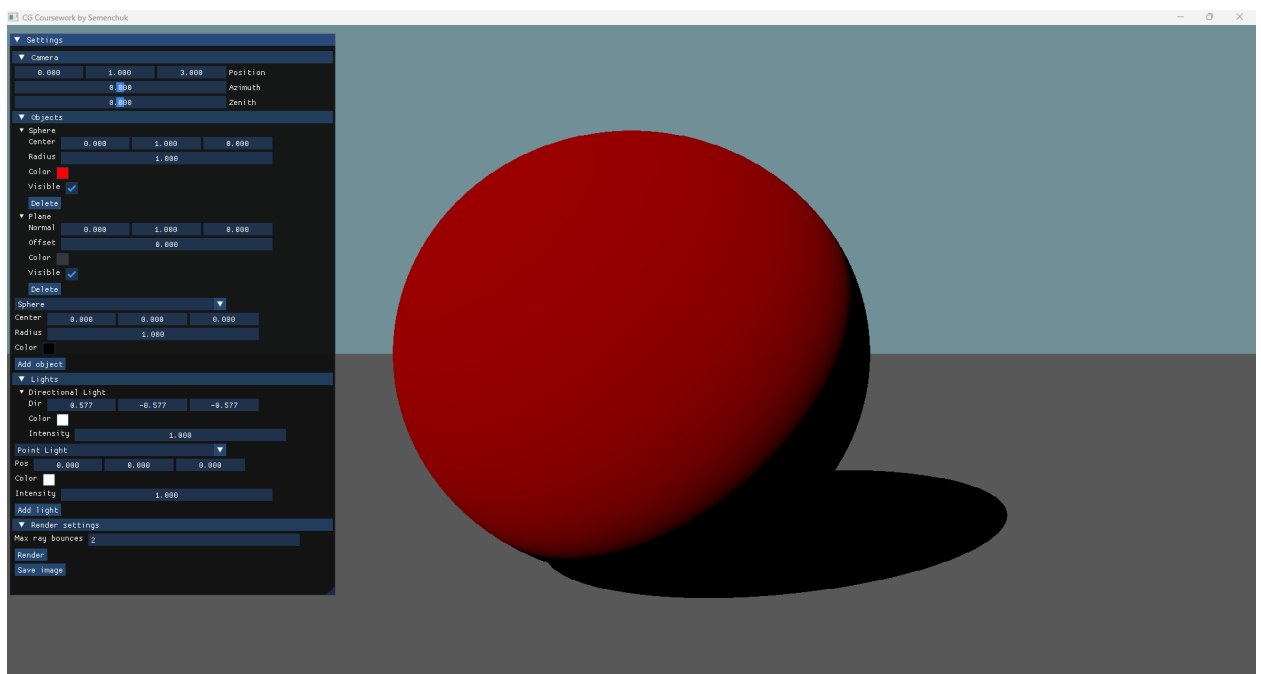


Рисунок 3.2 – Пример работы GUI

GUI позволяет:

- настраивать параметры камеры;

- добавлять, изменять и удалять геометрические примитивы и источники освещения;
- задавать параметры рендеринга, включая максимальное число отражений луча;
- сохранять сгенерированный кадр на устройство.

Для добавления геометрических примитивов или источников освещения пользователь должен выбрать соответствующий тип объекта в выпадающем меню, задать необходимые параметры и нажать кнопку Add Object или Add Light.

Также была реализована возможность управления положением и ориентацией камеры с помощью клавиатуры, аналогично большинству 3D-игр. Для перемещения камеры используются клавиши W (вперед), S (назад), A (влево), D (вправо), Page Up (подъём вверх) и Page Down (опускание вниз). Ориентация камеры управляется с помощью стрелок: Arrow Up и Arrow Down – наклон вверх и вниз, Arrow Left и Arrow Right – поворот влево и вправо.

Сохранение сгенерированного кадра осуществляется с помощью кнопки «Save image».

4 Исследовательский раздел

4.1 Анализ производительности ПО

В качестве тестовой сцены для анализа времени генерации кадра был выбран чайник университета Юта (Utah Teapot)[21], помещенный в Корнеллскую коробку (Cornell Box)[22]. Для освещения был использован точечный источник, помещенный на середину потолка.

Тестовая сцена состоит из:

- Корнеллской коробки;
- чайника Юта;
- точечного источника освещения.

Параметра генерации кадра:

- разрешение кадра 1280×720 пикселей;
- максимальное число переотражений луча = 2.

Параметры системы, на которой будут проводиться замеры времени:

- CPU – 6C/12T @ 3.7GHz;
- RAM – 32GB DDR5;
- OS – Windows 11.

Для распараллеливания программы использовался стандарт OpenMP[23]. Выбор обусловлен относительной простотой способа применения технологии. В языке C++ эта технология реализуется с помощью директивы `#pragma`.

Для установки числа потоков используется функция `omp_set_num_threads(int)` из заголовочного файла `<omp.h>`. Для получения количества логических ядер процессора используется функция `omp_get_max_threads()`.

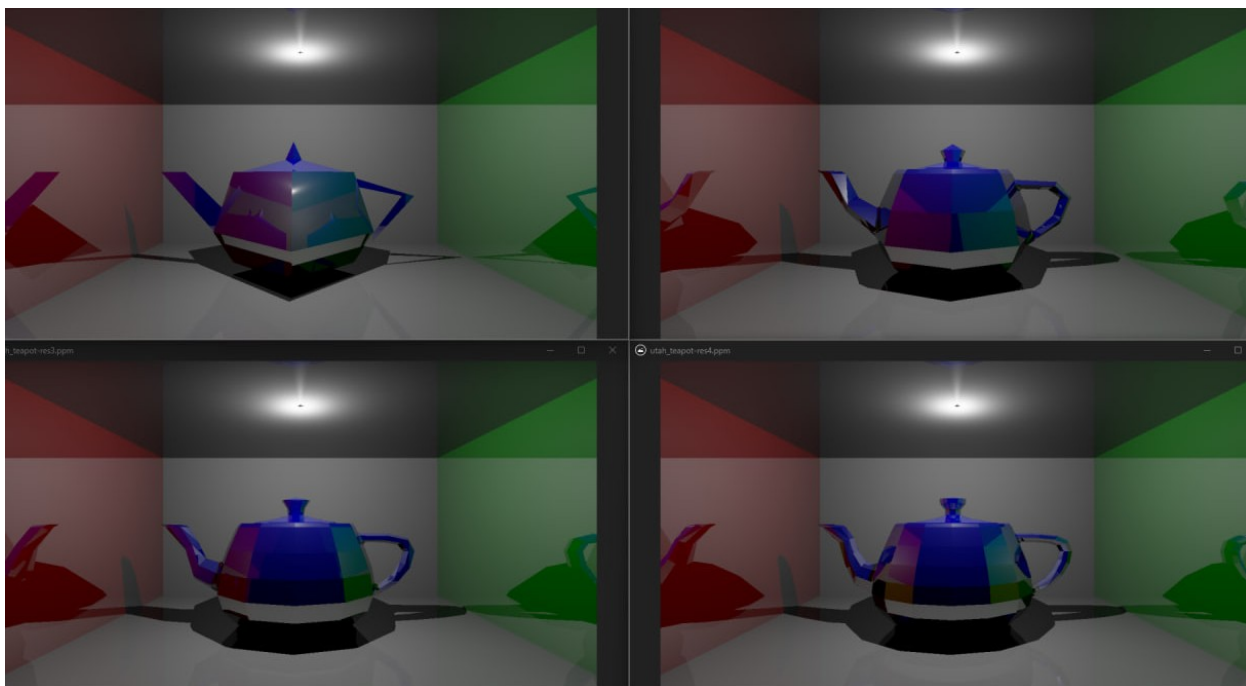


Рисунок 4.1 – Чайники с разным числом полигонов

Для исследования времени генерации кадра были выбраны следующие варианты числа потоков:

- Однопоточный режим (1);
- Количество потоков, равное числу физических ядер процессора (6);
- Количество потоков, равное числу логических ядер процессора (12);
- Вдвое большее количество потоков по сравнению с числом логических ядер процессора (24).

Следующие графики представляют собой зависимость времени генерации изображения от количества граней в полигональной модели чайника.

Таблица 4.1 – Время генерации кадра в зависимости от числа потоков и числа граней

Число граней	1 поток	6 потока	12 потоков	24 потоков
108	15.4	4.5	2.5	2.6
532	53.7	15.8	8.8	9.6
1166	114.7	30.7	18.8	18.6
2148	199.8	55.8	38.6	36.1

На графике ниже представлена зависимость времени генерации кадра от числа потоков.

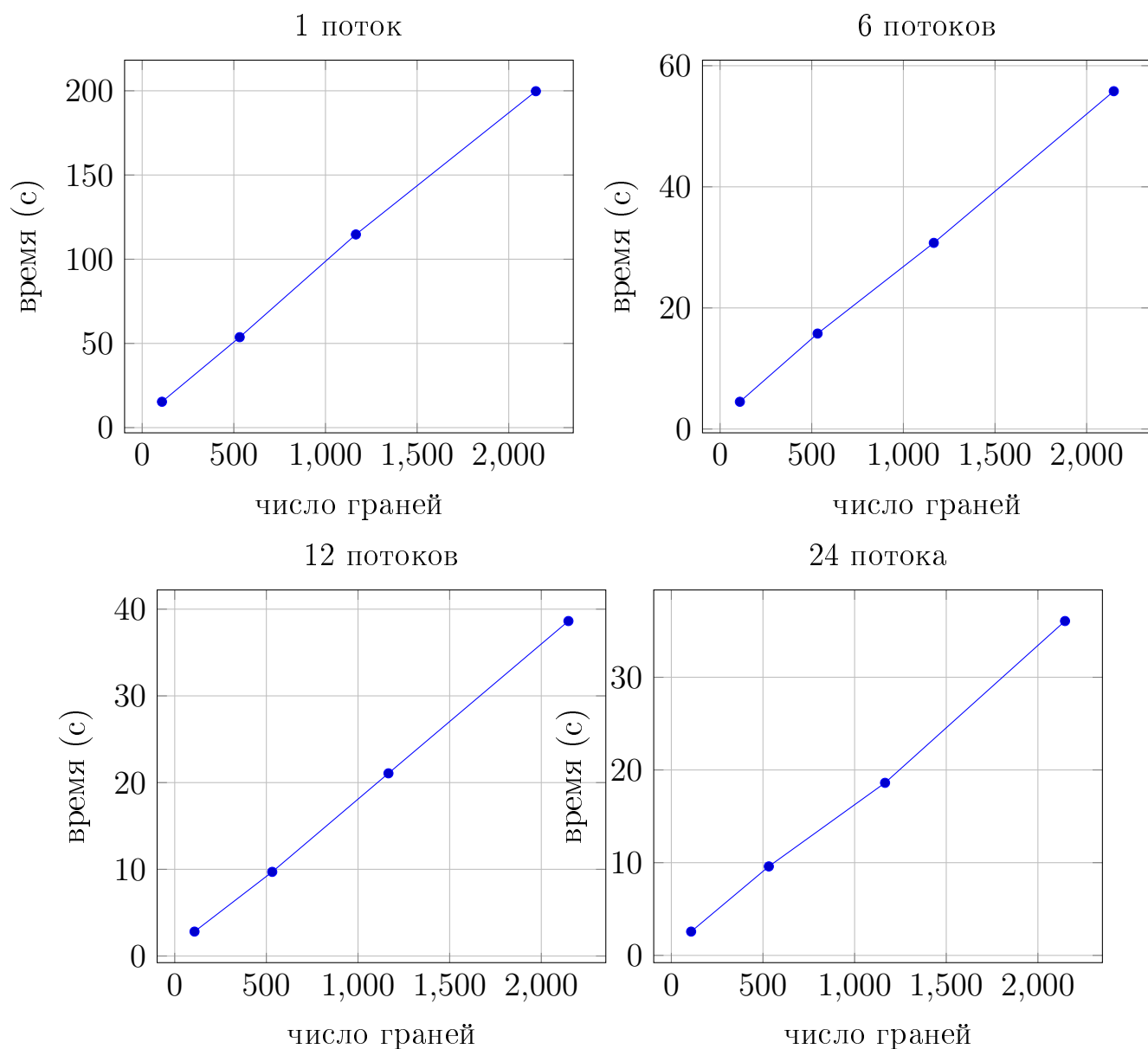
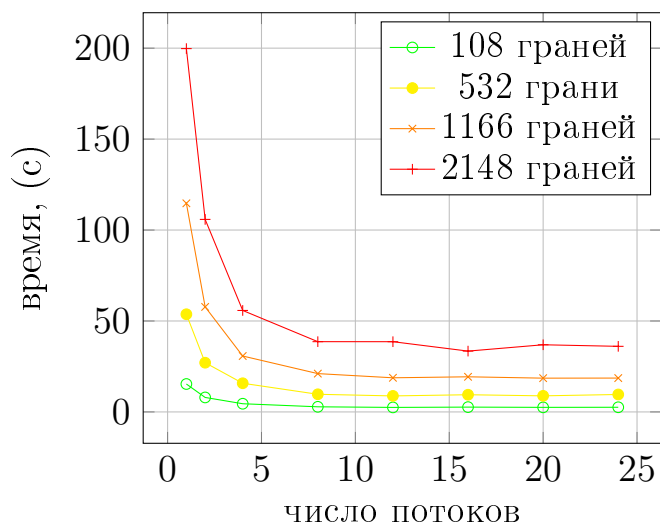


Рисунок 4.2 – Время генерации кадра в зависимости от числа потоков и числа граней

Время генерация кадра в зависимости от числа потоков



Выводы

Анализ представленных графиков позволяет сделать вывод, что прирост производительности с увеличением числа потоков носит ограниченный характер. Наиболее эффективным числом потоков является количество логических ядер процессора, при котором достигается максимальная параллельная загрузка. При дальнейшем увеличении числа потоков наблюдается снижение производительности, что связано с дополнительными затратами процессорного времени на переключение контекста между потоками.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы все поставленные задачи были успешно решены, и цель работы достигнута. Разработанное ПО позволяет отображать сцены с геометрическими примитивами, поддерживает добавление и удаление объектов и источников света, позволяет изменять их положение и ориентацию, а также управлять положением и направлением камеры. Реализованный алгоритм трассировки лучей в сочетании с графическим интерфейсом обеспечивает корректную визуализацию сцены и удобство взаимодействия пользователя с программой.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. *NVIDIA*. Ray Tracing article. — URL: <https://developer.nvidia.com/discover/ray-tracing>.
2. *Scratchapixel*. Диаграмма прямой трассировки лучей. — URL: <https://www.scratchapixel.com/images/introduction-to-ray-tracing/lighttoeyebounce.png> ; Изображение.
3. *Ландсберг Г. С.* Оптика. — Москва : Наука, 1981.
4. *Cloudfront*. Диаграмма обратной трассировки лучей. — URL: <https://d29g4g2dyqv443.cloudfront.net/sites/default/files/pictures/2018/RayTracing/ray-tracing-image-1.jpg> ; Изображение.
5. *Rogers D. F.* Procedural Elements for Computer Graphics. — 2nd. — New York : McGraw-Hill, 2001.
6. *Cloudfront*. Схема диффузного отражения Ламберта. — URL: <https://raytracing.github.io/images/fig-1.10-random-vec-horizon.jpg> ; Изображение.
7. *Vries J. de*. Basic Lighting. — URL: <https://learnopengl.com/Lighting/Basic-Lighting>.
8. *Vries J. de*. Phong diagram. — URL: https://learnopengl.com/img/advanced-lighting/advanced_lighting_over_90.png ; Изображение.
9. *Vries J. de*. Components of Phong lighting. — URL: https://learnopengl.com/img/lighting/basic_lighting_phong.png ; Изображение.
10. *Vries J. de*. Advanced Lighting. — URL: <https://learnopengl.com/Advanced-Lighting/Advanced-Lighting>.
11. *Vries J. de*. Изображение: Halfway Vector (Advanced Lighting). — URL: https://learnopengl.com/img/advanced-lighting/advanced_lighting_halfway_vector.png ; Изображение.

12. *Peter Shirley Trevor David Black S. H.* Ray Tracing in One Weekend. — 2025. — URL: <https://raytracing.github.io/books/RayTracingInOneWeekend.html>.
13. Ray Tracing Gems: High-Quality and Real-Time Rendering with DXR and Other APIs / под ред. E. Haines, T. Akenine-Möller. — Apress, 2019. — URL: <https://www.realtimerendering.com/raytracinggems/rtg/index.html>.
14. *Möller T., Trumbore B.* Fast, Minimum Storage Ray-Triangle Intersection // Journal of Graphics Tools. — 1997. — Т. 2, № 1. — С. 21–28. — DOI: 10.1080/10867651.1997.10487468.
15. *Pharr M., Jakob W., Humphreys G.* Physically Based Rendering: From Theory to Implementation. — URL: <https://www.pbr-book.org/>.
16. SFML — Simple and Fast Multimedia Library. — URL: <https://www.sfml-dev.org>.
17. GLFW — OpenGL Framework. — URL: <https://www.glfw.org>.
18. SDL — Simple DirectMedia Layer. — URL: <https://www.libsdl.org>.
19. Dear ImGui — Immediate Mode Graphical User Interface. — URL: <https://github.com/ocornut/imgui>.
20. CMake — Cross-Platform Build System. — URL: <https://cmake.org>.
21. *Utah Graphics Lab U. of.* The Utah Teapot. — URL: <https://graphics.cs.utah.edu/teapot/>.
22. *Program of Computer Graphics C. U.* Cornell Box. — URL: <https://graphics.cornell.edu/online/box/>.
23. *Board O. A. R.* OpenMP Application Programming Interface. — URL: <https://www.openmp.org/>.

ПРИЛОЖЕНИЕ А

К работе прилагается презентация, состоящая из 12 слайдов.